

System Architecture

Project 1: Minesweeper System Development — EECS 581: Software Engineering II
Fall 2026 · Professor Hossein Saiedian

1. Purpose

This document describes the high-level architecture of the Minesweeper implementation in Python. The program is a console-based Minesweeper game using a 10x10 grid. It explains the main components of the system, how data moves through the program, and the main data structures used to store the game information.

2. Components

- **Board** (board.py) : Defines the grid's data structure.
 - Cell class : an object that stores is_mine, is_uncovered, is_flagged, and adjacent_mines for a single cell
- **Game Logic** (game_logic.py) : Handles the rules of the game.
 - get_mine_count() : prompts and validates the player's chosen mine count (10–20).
 - place_mines() : randomly places mines on the grid without duplicates.
 - make_first_move_safe() : relocates a mine off the player's first uncovered cell, if needed.
 - count_adjacent_mines() : calculates each cell's adjacent mine count.
 - uncover_cell() / create_first_move_handler() : uncover a cell (with flood-fill for empty areas), wrapping the first move safely.
 - flag_cell() : toggles a flag on a covered cell.
 - count_flags() / remaining_mines() : track how many mines are still unflagged.
 - check_game_status() : determines "Playing," "Victory," or "Game Over: Loss."
- **Display** (display.py) : Displays the Minesweeper board, mine count, and game status to the player through the console.
 - print_grid() : renders the current board state (covered/flagged/uncovered cells and mine counts) to the console.
- **Player Input** (player_input.py) : Gets the player's input and uses it to perform an action on the selected cell.

- `get_player_input()` : parses commands like A4 (uncover) or A4, Flag (flag) and calls the corresponding action.
- `show_mines()` : reveals all mine locations after a loss.

3. Data Flow

The program first creates the 10x10 grid and gets the number of mines from the user. The mines are then placed randomly on the grid, and the adjacent mine counts are calculated. The program displays the board and waits for the player's input.

The program enters a loop where it displays the board, mines remaining, and current status, then waits for the player's next input.

The user's input is then parsed into an uncover or flag action and applied to the grid :the first uncover triggers a safety check that relocates a mine away from that cell if needed.

After each action, the game status is re-checked; the loop continues while the status is "Playing." Once the status becomes "Victory" or "Game Over: Loss," the final board is displayed, and if the player loses, all mine locations are revealed on the grid

4. Key Data Structures

Grid : a 2D list (`list[list[Cell]]`) of size $n \times n$, where n is fixed at 10. Represents the entire board state.

Cell : an object with four fields: `is_mine` (bool), `is_uncovered` (bool), `is_flagged` (bool), and `adjacent_mines` (int, 0–8).

5. Assumptions

Grid size fixed at 10x10. Mine count is user-specified within 10–20 via console input (`get_mine_count()`), validated with a retry loop for both non-numeric input and out-of-range values. The program checks the input and asks the user again if the value is not a number or is outside the range. Mines are randomly placed on the board without placing multiple mines in the same cell.